DecisionLab

Weekly Mini Project 03 — Team 2 · Project Specification

Project	DecisionLab
Team leader	Fadi Habil
Team members	Nagham Jaza, Iyad Ibrahim, Hamid Al Haj, Yousef Yousef, Majed Hmoud

> Stop guessing. Weigh what matters, score your options, and let the numbers explain the winner.

1. Project Overview

Hard choices — which laptop to buy, which city to move to, which project idea to build — are usually made by gut feeling. Important factors get forgotten, and it is hard to explain afterwards why one option won.

DecisionLab is a weighted decision engine. The user describes a decision, lists the options, defines the criteria that matter, gives each criterion an importance (weight), and scores every option on every criterion. The application calculates a weighted result, ranks the options and explains exactly why each option ended up where it did.

The value of the product is in the **calculation and its explanation**. The result must always be calculated by the application from the stored weights and scores — never typed in by the user.

2. Team

The team leader and members are listed in the table at the top of this document.

Team Responsibility

- The leader coordinates the internal work split and makes sure the team reaches a working integrated result.
- Frontend/backend ownership is not assigned by the instructor.
- The team should decide how responsibilities are distributed.
- Every member must make a meaningful contribution.
- Every member should understand the project at a high level.
- Every member must participate in the final English presentation.

3. Project Goal

Build a web application where a user can:

- create a decision, either blank or from a ready-made template;
- add options and criteria, and give each criterion a weight;
- score every option on every criterion;
- see a ranked, explained result that updates whenever weights or scores change;
- finalize a decision, which locks it, and reopen it if needed.

The weighted calculation, ranking and explanation are performed by the backend from the data stored in MongoDB.

4. Technology Constraints

Area	Requirement
Frontend	React + TypeScript
Backend	NestJS
Database	MongoDB, through Mongoose
API documentation	Swagger / OpenAPI

- All application data is stored in MongoDB and must survive a backend restart.
- How you organize your React app and your NestJS backend is your team's decision.
- Visual results (bars, highlights) must be built with normal React and CSS — no chart library is required.
- Do **not** add user accounts, login or passwords, another database, external AI services, cloud services or paid APIs.

5. User Scenario

Omar needs a laptop for the bootcamp. He creates the decision "Which laptop should I buy?" from the **Laptop** template, which gives him five criteria:

Criterion	Weight
Performance	30
Battery	25
Price	20
Portability	15
Display	10

He adds three options and starts scoring from 1 to 10 (10 is always best for him — a cheaper laptop gets a higher Price score). When he asks for the result before scoring everything, DecisionLab tells him exactly which scores are missing.

Once all scores are in, the result ranks the options. Option A wins with **78.5**; the explanation shows that most of its points come from Performance and Battery, and that it is 6.0 points ahead of Option B.

He raises Price to 30 and lowers Performance to 20. The total is now 100 again and the ranking changes: Option B moves to first place. Happy with this, he finalizes the decision; it is now locked.

6. Required Features

Feature	Description
Decisions list	All decisions with title, status and last update; filter by status, search by title.
Create decision	Create a blank decision, or one based on a template.
Options	Add, rename and remove options.
Criteria and weights	Add, rename and remove criteria, and set each criterion's weight.
Weight total	Always show the current total of all weights and how far it is from 100.
Score matrix	Enter a score for every option on every criterion.
Result	Ranked options with weighted totals and an explanation per option.
Finalize / reopen	Lock a complete decision; reopen it to edit again.

Feature	Description
Templates	Browse ready-made templates; templates are managed internally.
Delete decision	Remove a decision and everything that belongs to it.

Decision information

Each decision must have at least: title, optional description, status (**Draft** or **Finalized**), its options, its criteria with weights, its scores, the date it was created and last updated, and — once finalized — the date it was finalized.

Template information

Each template must have at least: name, a short description and a list of criteria with weights that total 100.

7. Business Rules

Options and criteria

1. A decision has between **2 and 8 options** and between **2 and 8 criteria** before it can be calculated or finalized.
2. Option names are unique within a decision, and criterion names are unique within a decision (ignoring upper/lower case and surrounding spaces).
3. Removing an option or a criterion also removes all of its scores.

Weights

1. A weight is a whole number from **1 to 100**.
2. While a decision is a Draft, the weights may temporarily total something other than 100. The application always shows the current total.
3. A result can only be calculated, and a decision can only be finalized, when the weights total **exactly 100**.

Scores

1. A score is a whole number from **1 to 10**, where 10 always means "best for me" on that criterion.
2. A result can only be calculated when **every option has a score on every criterion**. Otherwise the request is rejected, listing every missing option/criterion pair.

Calculation

1. An option's weighted total is the sum of **score × weight** over all criteria, divided by **10**. The result is between 10 and 100.
2. Each criterion's **contribution** to an option is that criterion's score × weight ÷ 10.
3. Totals are shown with one decimal place. Ranking uses the exact, unrounded totals.
4. Options are ranked from the highest total to the lowest.
5. **Ties:** if two options have exactly the same total, the one with the higher score on the criterion with the highest weight ranks first. If they are still tied, they share the same rank and are clearly marked as tied.
6. The same data must always produce exactly the same result and order.
7. The result is always calculated from the current stored weights and scores. A stored result that is not recalculated after a change is not acceptable.

Finalize and templates

1. Finalizing requires all the conditions above to be met. The finalized decision records its winner (or the tied winners) and the finalize date.
2. A Finalized decision is **locked**: changing options, criteria, weights or scores is rejected until the decision is reopened. Reopening returns it to Draft.

3. Creating a decision from a template **copies** the template's criteria and weights. Later changes to the template must not change existing decisions.
4. Creating, editing and deleting templates are internal operations (see section 13). Template weights must total exactly 100.

8. Required Screens / Views

Screen	Must show / allow
Decisions	All decisions with status and last update; status filter and title search; create and delete actions.
New Decision	Title and description; choose "blank" or a template.
Decision Workspace	Options, criteria with weights, the live weight total (with a clear warning when it is not 100), and the score matrix with missing scores highlighted.
Result	Ranked options with rank, total and a visual bar; per-option explanation (contribution of each criterion, strongest and weakest criterion, gap to the option above); finalize / reopen actions.
Templates	Available templates with their criteria and weights, and a way to start a decision from one.

All screens must be reachable through the application's navigation.

9. Required User Flows

Flow A — Set up a decision

1. The user creates a decision from the Laptop template.
2. The criteria and weights are copied into the new decision.
3. The user adds three options.

Flow B — Incomplete data is reported

1. The user scores only some of the options.
2. The user asks for the result.
3. The request is rejected, and the user sees exactly which option/criterion scores are missing.
4. The user changes one weight so that the total becomes 105 and asks again: the user is told the weights must total 100.

Flow C — See and understand the result

1. The user completes the scores and fixes the weights.
2. The Result screen shows the ranked options with their totals.
3. The user opens an option's explanation and sees how many points each criterion contributed.

Flow D — Change and re-rank

1. The user moves weight from Performance to Price, keeping the total at 100.
2. The result is recalculated and the ranking changes accordingly.

Flow E — Finalize and reopen

1. The user finalizes the decision; the winner and the date are recorded.
2. Trying to change a score is rejected because the decision is locked.
3. The user reopens the decision, which becomes a Draft again and can be edited.

10. Required Calculations / Derived Information

Derived information	Where it appears
Current weight total and difference from 100	Decision Workspace
List of missing scores	Decision Workspace, Result (when rejected)
Weighted total per option (one decimal)	Result
Rank per option, including shared ranks for ties	Result
Contribution of each criterion to each option	Result
Strongest and weakest criterion of each option	Result
Gap between each option and the option ranked directly above it	Result
Winner(s) of a finalized decision	Decisions, Result

11. Validation & Error Scenarios

General expectations

- Required information cannot be missing; numbers must be sensible.
- Invalid actions must be rejected — the system must never silently accept an invalid operation.
- The user always receives a clear, human-readable message explaining **why** an action failed.
- Something that does not exist is reported as "not found", not as an empty success.
- A request that conflicts with the current data (for example a duplicate or a locked decision) is reported as a conflict.
- When a request cannot be completed, the screen shows a sensible message — never a blank page.

Project-specific scenarios

Situation	Expected behavior
Decision title missing or only spaces	Rejected.
Duplicate option or criterion name in the same decision	Rejected as a conflict.
More than 8 options or criteria	Rejected.
Weight is 0, negative, above 100 or not a whole number	Rejected.
Score is outside 1–10 or not a whole number	Rejected.
A score for an option or criterion that is not part of this decision	Rejected.
Calculating while weights do not total 100	Rejected; the current total is reported.
Calculating with missing scores	Rejected; every missing pair is listed.
Calculating with fewer than 2 options or criteria	Rejected.
Any change to a Finalized decision	Rejected as a conflict until reopened.
A decision or template that does not exist	"Not found".
A template whose weights do not total 100	Rejected.

12. Data & Persistence Requirements

- All decisions and templates are stored in MongoDB.

- On first start with an empty database, the backend seeds at least **3 templates** (for example Laptop, City to live in, Project idea) and at least **2 example decisions**: one complete Draft and one Finalized.
- Records are identified and linked using the **numeric, application-level id convention used in class** (as in B4F Board).

Data Design Decisions

Your team must make, implement and be able to explain these decisions:

1. **Relationship decision.** Decide where options, criteria and scores live: inside the decision document, or in separate collections linked by id. Decide also how a decision remembers which template it came from. Write down your choices and why they fit how the data is read and changed.
2. **Index decision.** Choose one real, frequently used MongoDB query from your Decisions screen whose filter and sort fields are stored in your documents — for example, decisions with a given status, most recently updated first, if that is your actual query shape. Measure it with `explain("executionStats")`, choose an index only if the measurement and the query shape justify it, then measure again to prove the result.
    - Title search can remain part of the API, but do not assume an index on the title automatically improves a partial, case-insensitive search.
    - Choose and prove the index from the actual measured query shape: query shape → measurement → index decision → measurement again.

Both decisions must be written in your repository's README.

13. Backend Quality Requirements

- **Validation:** every request body and every filter query parameter is validated with DTOs and the validation approach taught in class. Unknown fields are rejected.
- **Meaningful errors:** failures use the correct kind of HTTP error (bad request, not found, conflict) with a clear message.
- **Logger:** log the events that matter — decisions created from templates, finalize and reopen (with the winner), and rejected calculations with the reason. Do **not** log every request.
- **Swagger:** every endpoint is documented with a short summary, grouped by resource, with its request body and query parameters visible.
- **Internal-only actions:** creating, editing and deleting **templates** must be protected with the `InternalToolGuard` approach taught in class (documented in Swagger, including the required header). Reading templates and everything a user does with their own decisions must **not** be guarded.
- **No fake logic:** totals, ranks and explanations must come from real calculations on stored data. Hard-coded or hand-typed results do not count.

14. Out of Scope

The following are **not required** and must not be added to the core project: user accounts, login, passwords or JWT; sharing decisions between users; Mongoose ObjectId references or `populate`; aggregation pipelines; pagination; WebSockets; chart libraries; another database; external AI services; deployment.

15. Minimum Deliverable (CORE)

Your project counts as complete only when **all** of the following work:

- □ A working React + TypeScript frontend and a working NestJS backend, integrated with each other.
- □ Data is stored in MongoDB and survives a backend restart; templates and example decisions are seeded.
- □ Every required screen in section 8 exists and is reachable through the navigation.
- □ Every business rule in section 7 is enforced.

- □ Decisions can be created blank or from a template, edited and deleted.
- □ Weight totals, missing-score reporting and the 100-total rule work.
- □ Totals, ranks, tie handling and explanations are correct and update after every change.
- □ Finalize locks the decision; reopen unlocks it.
- □ Templates are managed only through guarded internal endpoints.
- □ Validation, error messages, Logger and Swagger meet section 13.
- □ Both data design decisions are implemented and written in the README.
- □ The README explains how to install, configure (environment variables) and run both parts.

16. Stretch Features (optional)

Only start these after every CORE item works. They never replace a missing CORE requirement.

- **What-if analysis** — temporarily change one weight and immediately see how the ranking would change, without saving the change.
- **Sensitivity hint** — for the winner, show how much one weight would need to change for the second option to take first place.
- **Duplicate decision** — copy a decision to try a different set of weights.
- **Template management screen** — an internal screen that sends the internal-tool header.

17. Presentation & Demo Acceptance

Your team presents **in English**; every member speaks. Your instructor will confirm the date and time limit. Present the problem, a short product walkthrough, a live demo, your data design decisions and the most important business rules.

The live demo must pass these acceptance tests:

1. Creating a decision from a template copies its criteria; editing the template afterwards does not change that decision.
2. Calculating with missing scores is rejected and lists the missing pairs.
3. Calculating with weights totalling 105 is rejected and shows the total.
4. With the weights from the scenario, one option's total can be checked by hand and matches the application.
5. Changing two weights (still totalling 100) changes the ranking, and the explanation updates.
6. Two options with identical totals are ranked using the tie rule.
7. A Finalized decision rejects changes; after reopening, changes are accepted.
8. In Swagger, creating a template without the internal-tool header is rejected, and with it succeeds.
9. Restarting the backend keeps all data.

18. Evaluation Focus

Area	What we look for
Functional completeness	The required features and screens exist and work.
Correct business behavior	Calculation, ranking, ties and locking follow the rules exactly.
Frontend/backend integration	The two parts genuinely work together on real stored data.
Backend quality	Validation, meaningful errors, Logger, Swagger and correct guard placement.
Data design	Sensible, explained relationship and index decisions.
User experience	Clear score matrix and an understandable explanation of the result.

Area	What we look for
Team collaboration and Git	Shared work, visible contributions from every member.
Explaining your own work	Every member can explain the product, its rules and its design.
Presentation quality	Clear English presentation with a working live demo.